Ship Scheduling Model Optimization with Gurobi

This Jupyter notebook describes a ship scheduling optimization model implemented using Gurobi. The model assigns ships to berths and schedules their unloading in such a way that minimizes the total cost of waiting, berth capacity increases, and unloading time. The goal is to efficiently manage port resources, particularly berths, to optimize the ship unloading process.

Key Objectives:

1. Minimize the cost of waiting for ships.
2. Minimize the additional capacity increases required for berths.
3. Minimize the unloading time for ships.

The problem is solved using Mixed Integer Linear Programming (MILP) and Gurobi optimization.

```
from gurobipy import *
```

------------------ Data Definition ------------------

```
# Ships: List of ships in the system
Ships = ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8']
# Berths: List of berths available at the port
Berths = ['B1', 'B2', 'B3', 'B4']
# M: Large number used for big-M method in constraints
M = 1e9
# epsilon: A small value to represent a very small threshold
epsilon = 1e-9
# max_waiting_without_priority: Maximum waiting time for ships without
priority
max_waiting_without_priority = 80

# Ship parameters: Cargo size, length, priority, and arrival time for
each ship
shipCargoSize = {
    "S1": 6,  "S2": 18, "S3": 20, "S4": 5,
    "S5": 11, "S6": 19, "S7": 20, "S8": 13,
}

shipLength = {
    "S1": 114, "S2": 126, "S3": 115, "S4": 102,
    "S5": 102, "S6": 131, "S7": 119, "S8": 138,
```

```
}

shipPriority = {
    "S1": 0.9638, "S2": 0.7731, "S3": 0.4415, "S4": 0.9819, "S5":
0.1909, "S6": 0.3487, "S7": 0.9705, "S8": 0.4456,
}

shipArrivalTime = {
    "S1": 4.7655, "S2": 1.0046, "S3": 3.6520, "S4": 6.0542, "S5":
1.8692, "S6": 2.1180, "S7": 4.6230, "S8": 6.6012,
}
```

---------- Berth Parameters ----------

```
# Berth parameters: Number of available anchors, capacity, length,
lag, and cost to increase capacity for each berth
berthNumberOfAnchs = {
    "B1": 2, "B2": 2, "B3": 3, "B4": 2,
}

berthCapacity = {
    "B1": 57, "B2": 40, "B3": 52, "B4": 30,
}

berthLength = {
    "B1": 118, "B2": 236, "B3": 120, "B4": 138,
}

berthLag = {
    "B1": 6.9221, "B2": 9.7278, "B3": 9.5648, "B4": 5.7483,
}

berthIncreaseCapCost = {
    "B1": 5.8715, "B2": 5.8804, "B3": 8.8642, "B4": 6.6403,
}

berthMaxCapacityIncrease = {
    "B1": 29, "B2": 15, "B3": 49, "B4": 17,
}
```

---------- Ship Priority Matrix ----------

```
# Matrix to define priority between ships (1 indicates higher
priority)
shipPriorityMatrix = {(i, k): 0 for i in Ships for k in Ships}
# Updating matrix with values where ships have priority over others
shipPriorityMatrix.update({
    ("S1", "S2"): 1, ("S1", "S3"): 1, ("S1", "S5"): 1, ("S1", "S6"):
1, ("S1", "S8"): 1,
    ("S2", "S3"): 1, ("S2", "S5"): 1, ("S2", "S6"): 1, ("S2", "S8"):
1,
    ("S3", "S5"): 1, ("S3", "S6"): 1,
    ("S4", "S1"): 1, ("S4", "S2"): 1, ("S4", "S3"): 1, ("S4", "S5"):
1, ("S4", "S6"): 1, ("S4", "S7"): 1, ("S4", "S8"): 1,
    ("S6", "S5"): 1,
    ("S7", "S1"): 1, ("S7", "S2"): 1, ("S7", "S3"): 1, ("S7", "S5"):
1, ("S7", "S6"): 1, ("S7", "S8"): 1,
    ("S8", "S3"): 1, ("S8", "S5"): 1, ("S8", "S6"): 1,
})
```

---------- Unloading Time for Ships at Each Berth ----------

```
# Time it takes to unload each ship at each berth
unloadShipInAnchTime = {
    ("S1", "B1"): 9.31, ("S1", "B2"): 8.32, ("S1", "B3"): 9.04, ("S1",
"B4"): 8.66,
    ("S2", "B1"): 8.50, ("S2", "B2"): 9.91, ("S2", "B3"): 9.99, ("S2",
"B4"): 8.09,
    ("S3", "B1"): 9.72, ("S3", "B2"): 9.21, ("S3", "B3"): 8.76, ("S3",
"B4"): 8.57,
    ("S4", "B1"): 9.35, ("S4", "B2"): 8.91, ("S4", "B3"): 9.37, ("S4",
"B4"): 9.32,
    ("S5", "B1"): 8.27, ("S5", "B2"): 9.54, ("S5", "B3"): 9.96, ("S5",
"B4"): 9.94,
    ("S6", "B1"): 9.23, ("S6", "B2"): 8.09, ("S6", "B3"): 8.01, ("S6",
"B4"): 8.27,
    ("S7", "B1"): 9.88, ("S7", "B2"): 8.61, ("S7", "B3"): 8.73, ("S7",
"B4"): 9.80,
    ("S8", "B1"): 8.63, ("S8", "B2"): 9.10, ("S8", "B3"): 8.87, ("S8",
"B4"): 8.13,
}
```

---------- Model Initialization ----------

```
# Create optimization model
model = Model('Ship_Berth_Assignment')
```

---------- Variable Definition ----------

```
# Define variables for berth capacity increase, ship waiting times,
and ship assignments
berthCapIncrease = model.addVars(Berths, vtype=GRB.INTEGER, lb=0,
name='berthCapIncrease')
shipWaitingTime = model.addVars(Ships, vtype=GRB.CONTINUOUS, lb=0,
name='shipWaitingTime')
shipWaitingTime_parts = model.addVars([1, 2, 3], Ships,
vtype=GRB.CONTINUOUS, lb=0, name='waiting_parts')
shipWaitingTime_binary_helpers = model.addVars([1, 2], Ships,
vtype=GRB.BINARY, name='waiting_binary_helpers')
shipCostOfWaiting = model.addVars(Ships, vtype=GRB.CONTINUOUS, lb=0,
name='shipCostOfWaiting')
shipAssignment = model.addVars(Ships, Berths, vtype=GRB.BINARY,
name='shipAssignment')
shipUnloadStart = model.addVars(Ships, vtype=GRB.CONTINUOUS, lb=0,
name='unloadStart')
shipiUnloadBeforekInBerth = model.addVars(Ships, Ships, Berths,
vtype=GRB.BINARY, name='iBeforek')
```

---------- Sequence Constraints ----------

```
# Add constraints to enforce the unloading sequence between ships at
each berth
for i in Ships:
    for k in Ships:
        if i == k:
            continue
        for j in Berths:
            model.addConstr(
                shipUnloadStart[k] >= shipUnloadStart[i] +
unloadShipInAnchTime[i, j] + berthLag[j] -
```

```
                M * (1 - shipiUnloadBeforekInBerth[i, k, j]),
                name=f"seq1_{i}_{k}_{j}"
            )
            model.addConstr(
                shipiUnloadBeforekInBerth[i, k, j] <=
shipAssignment[i, j],
                name=f"zact1_{i}_{k}_{j}"
            )
            model.addConstr(
                shipiUnloadBeforekInBerth[i, k, j] <=
shipAssignment[k, j],
                name=f"zact2_{i}_{k}_{j}"
            )
            model.addConstr(
                shipiUnloadBeforekInBerth[i, k, j] +
shipiUnloadBeforekInBerth[k, i, j] >=
                shipAssignment[i, j] + shipAssignment[k, j] - 1,
                name=f"zact3_{i}_{k}_{j}"
            )
            if Ships.index(i) < Ships.index(k):
                model.addConstr(
                    shipiUnloadBeforekInBerth[i, k, j] +
shipiUnloadBeforekInBerth[k, i, j] <= 1,
                    name=f"symm_{i}_{k}_{j}"
                )
            model.addConstr(
                shipiUnloadBeforekInBerth[i, k, j] >= -2 +
shipPriorityMatrix[i, k] + shipAssignment[i, j] + shipAssignment[k,
j],
                name=f"priority_{i}_{k}_{j}"
            )
```

------------------ Assignment Constraints ------------------

```
# Each ship should be assigned to exactly one berth
for i in Ships:
    model.addConstr(shipAssignment.sum(i, '*') == 1,
name=f"assign_{i}")
```

---------- Capacity Constraints ----------

```
# Ensure that the total cargo size assigned to each berth does not
exceed its capacity
for j in Berths:
    model.addConstr(
        quicksum(shipAssignment[i, j] * shipCargoSize[i] for i in
Ships) <= berthCapacity[j] + berthCapIncrease[j],
        name=f"cap_{j}"
    )
```

---------- Capacity Increase Constraints ----------

```
# Limit the capacity increase for each berth to its maximum allowable
increase
for j in Berths:
    model.addConstr(berthCapIncrease[j] <=
berthMaxCapacityIncrease[j], name=f"capIncMax_{j}")
```

---------- Anchor Constraints ----------

```
# Limit the number of ships that can be assigned to each berth based
on the available anchors
for j in Berths:
    model.addConstr(shipAssignment.sum('*', j) <=
berthNumberOfAnchs[j], name=f"anchMax_{j}")
# Ensure that berth B3 has at least the minimum number of anchors
assigned
model.addConstr(shipAssignment.sum('*', 'B3') >=
berthNumberOfAnchs['B3'], name="anchMin_B3")


<gurobi.Constr *Awaiting Model Update*>
```

---------- Length Constraints ----------

```
# Ensure that each ship fits within the assigned berth length
for i in Ships:
    for j in Berths:
        model.addConstr(shipAssignment[i, j] * shipLength[i] <=
berthLength[j], name=f"length_{i}_{j}")
```

---------- Timing Constraints ----------

```
# Ensure that each ship's unload start time is greater than or equal
to its arrival time
for i in Ships:
    model.addConstr(shipUnloadStart[i] >= shipArrivalTime[i],
name=f"after_arrival_{i}")
    model.addConstr(shipWaitingTime[i] == shipUnloadStart[i] -
shipArrivalTime[i], name=f"waiting_{i}")
    model.addConstr(shipWaitingTime[i] <= (1 - shipPriority[i]) *
max_waiting_without_priority, name=f"max_wait_{i}")
```

---------- Waiting Cost Constraints ----------

```
# Split waiting time into parts and calculate the cost of waiting
for i in Ships:
    w1 = shipWaitingTime_parts[1, i]
    w2 = shipWaitingTime_parts[2, i]
    w3 = shipWaitingTime_parts[3, i]
    b1 = shipWaitingTime_binary_helpers[1, i]
    b2 = shipWaitingTime_binary_helpers[2, i]
    model.addConstr(shipWaitingTime[i] == w1 + w2 + w3,
name=f"wait_sum_{i}")
    model.addConstr(shipCostOfWaiting[i] == 25*w1 + 10*w2 + 5*w3,
name=f"cost_{i}")
    model.addConstr(10 * b1 <= w1)
    model.addConstr(w1 <= 10)
    model.addConstr(30 * b2 <= w2)
    model.addConstr(w2 <= 30 * b1)
```

```
    model.addConstr(w3 <= 30 * b2)
```

------------------ Objective Function ------------------

```
# Minimize the total cost: berth capacity increase, ship waiting cost, and unloading time
model.setObjective(
    quicksum(berthIncreaseCapCost[j] * berthCapIncrease[j] for j in Berths) +
    quicksum(shipCostOfWaiting[i] for i in Ships) +
    5 * quicksum(shipAssignment[i, j] * unloadShipInAnchTime[i, j] for i in Ships for j in Berths),
    GRB.MINIMIZE
)
```

------------------ Optimization ------------------

```
# Solve the optimization model
model.optimize()

Gurobi Optimizer version 12.0.2 build v12.0.2rc0 (win64 - Windows 11.0 (22631.2))

CPU model: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz, instruction set [SSE2|AVX|AVX2|AVX512]
Thread count: 4 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 1365 rows, 356 columns and 3672 nonzeros
Model fingerprint: 0xc9246d77
Variable types: 48 continuous, 308 integer (304 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+09]
  Objective range  [1e+00, 5e+01]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+09]
Warning: Model contains large matrix coefficients
         Consider reformulating model or setting NumericFocus parameter
         to avoid numerical issues.
```

```
Presolve removed 1296 rows and 315 columns
Presolve time: 0.01s
Presolved: 69 rows, 41 columns, 214 nonzeros
Variable types: 10 continuous, 31 integer (30 binary)
Found heuristic solution: objective 1803.1778007

Root relaxation: objective 1.367452e+03, 33 iterations, 0.00 seconds
(0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |
Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap |
It/Node Time

     0     0 1367.45226    0   22 1803.17780 1367.45226  24.2%     -
0s
     0     0 1538.69220    0   22 1803.17780 1538.69220  14.7%     -
0s
     0     0 1545.92071    0   17 1803.17780 1545.92071  14.3%     -
0s
     0     0 1803.17780    0   18 1803.17780 1803.17780  0.00%     -
0s

Cutting planes:
  Gomory: 3
  Cover: 3
  Implied bound: 9
  MIR: 5
  Flow cover: 1
  Mod-K: 1
  RLT: 3
  Relax-and-lift: 1

Explored 1 nodes (73 simplex iterations) in 0.03 seconds (0.01 work
units)
Thread count was 8 (of 8 available processors)

Solution count 1: 1803.18

Optimal solution found (tolerance 1.00e-04)
Best objective 1.803177800716e+03, best bound 1.803177800716e+03, gap
0.0000%


# ------------------- Output Results -------------------
# Print the optimized results (only non-zero variables)
for v in model.getVars():
    if v.X > 1e-6:  # Only print variables that have non-zero values
        print(f"{v.VarName}: {v.X}")
```

```
berthCapIncrease[B4]: 1.0
shipWaitingTime[S3]: 19.265800072669983
shipWaitingTime[S5]: 39.37340006122591
shipWaitingTime[S6]: 20.6952999982834
shipWaitingTime[S8]: 8.241699975776681
waiting_parts[1,S3]: 10.0
waiting_parts[1,S5]: 10.0
waiting_parts[1,S6]: 10.0
waiting_parts[1,S8]: 8.241699975776644
waiting_parts[2,S3]: 9.265800072669983
waiting_parts[2,S5]: 29.373400061225908
waiting_parts[2,S6]: 10.695299998283398
waiting_binary_helpers[1,S3]: 1.0
waiting_binary_helpers[1,S5]: 1.0
waiting_binary_helpers[1,S6]: 1.0
shipCostOfWaiting[S3]: 342.6580007266998
shipCostOfWaiting[S5]: 543.734000612259
shipCostOfWaiting[S6]: 356.95299998283394
shipCostOfWaiting[S8]: 206.0424993944161
shipAssignment[S1,B2]: 1.0
shipAssignment[S2,B4]: 1.0
shipAssignment[S3,B3]: 1.0
shipAssignment[S4,B1]: 1.0
shipAssignment[S5,B3]: 1.0
shipAssignment[S6,B2]: 1.0
shipAssignment[S7,B3]: 1.0
shipAssignment[S8,B4]: 1.0
unloadStart[S1]: 4.7655
unloadStart[S2]: 1.0046
unloadStart[S3]: 22.917800072669984
unloadStart[S4]: 6.0542
unloadStart[S5]: 41.24260006122591
unloadStart[S6]: 22.813299998283398
unloadStart[S7]: 4.623
unloadStart[S8]: 14.842899975776682
iBeforek[S1,S6,B2]: 1.0
iBeforek[S2,S8,B4]: 1.0
iBeforek[S3,S5,B3]: 1.0
iBeforek[S7,S3,B3]: 1.0
iBeforek[S7,S5,B3]: 1.0
```